Capital structure arbitrage, explained

Three methods, one data layer, and what the evidence does and does not show

Plain-English and technical companion to the E2C repository

1 November 2026

In plain English

How to read this. Each section has a teal box and a blue box. The teal box says what the idea is and why it matters to you. The blue box gives the exact equations and the function that computes them. Orange boxes hold worked examples. Red boxes hold limits.

The one-line version. Stock and option prices imply a credit spread. The credit market quotes its own. When the two disagree, we trade one against the other and wait for them to agree.

The main result so far. The code runs and reproduces your workbook. No method has run on your production data. Every Sharpe ratio here is in-sample, and it comes from a panel that holds survivors only. None of them shows that the trade makes money.

Contents

1 The idea **2**

2 The shared engine: CreditGrades **3**

3 The workbook method **4**

4 Yu's method on bonds **6**

5 Finkelstein: options join in **7**

6 The debt-equity spread and F10 **9**

7 Selecting names: the hold band and the cap **10**

8 The data layer **13**

9 What is proven and what is not **14**

1 The idea

In plain English

The picture. A company's stock and its debt are claims on the same assets. If the stock market thinks the firm is safe and the credit market thinks it is risky, one of them is wrong.

The trade. You own the cheap claim and you are short the rich one.

- **Credit looks too wide** (the market spread sits above what equity implies). Own the credit: sell CDS protection or buy the bond. Short the stock as a hedge.
- **Credit looks too tight** (the market spread sits below what equity implies). Short the credit: buy CDS protection or short the bond. Buy the stock as a hedge.

Why it matters to you. Everything in this repository is one version of this comparison. The methods differ in how they compute the equity side, which credit instrument they use, and when they trade.

New terms. A **CDS** (credit default swap) is insurance on a firm's debt. The buyer pays a yearly **spread** and receives a payment if the firm defaults. A **hedge** is a second position that offsets the first position's risk.

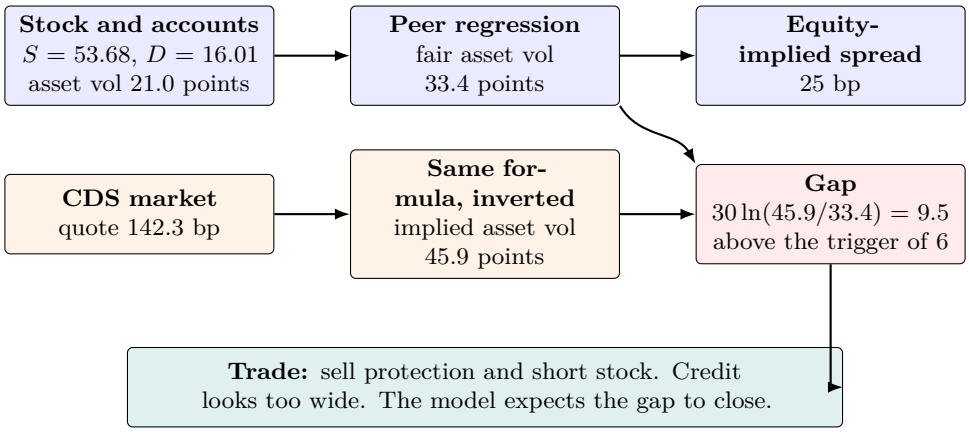


Figure 1: The whole method on one real row: Altria (MO), 16 January 2004. Blue is the equity side. Orange is the credit side. The gap decides the trade. Section 3 computes each number.

Technical detail

Let c be the market spread and c^* the spread implied by equity. A method chooses a gap measure g and a trigger τ.

$$g > \tau: \text{ long credit, short stock.} \qquad g < -\tau: \text{ short credit, long stock.}$$

The three methods choose as follows.

Method	Credit leg	Gap g	Trigger
Workbook	5-year CDS	$30 \ln(\sigma^{\text{imp}}/\sigma^{\text{fair}})$	± 6, weekly
Yu on bonds	Bond	$\ln(z^{\text{mkt}}/z^{\text{model}})$	$\ln(1+\alpha)$, $\alpha = 0.5$, daily
Rank book (DES)	Bond and stock	$CS_D - CS_E$, in bp	Top and bottom quintile, monthly

Finkelstein's deck supplies an option-calibrated engine and no rule. Section 5 covers it.

2 The shared engine: CreditGrades

In plain English

The picture. Treat the firm's assets as a random walk. The firm defaults the first time its assets touch a barrier set by its debt. The barrier's height is uncertain, because nobody knows how much of the debt holders will recover until the default comes.

What you put in and get out. You put in the stock price, debt per share and asset volatility. You get out the chance of surviving to any date. That chance gives a fair CDS spread.

Why it matters to you. All your methods share this one formula. So they can share its errors. A disagreement between methods does not prove that one of them is right.

New terms. Asset volatility (asset vol) is how much the whole firm's value moves in a year. It sits below equity vol, because debt magnifies moves in the stock. A **basis point** (bp) is 0.01%.

Equations and code

Symbols. S stock price. D debt per share. $\bar{L}$ mean barrier as a share of debt. λ uncertainty of the barrier. σ_A asset vol. σ_S equity vol. T maturity. r risk-free rate. R recovery. Φ the standard normal distribution function.

Asset vol from equity vol.

$$\sigma_A = \sigma_S \, \frac{S}{S + \bar{L}D}$$

Survival. Set

$$d = \Big(1 + \frac{S}{\bar{L}D}\Big) e^{\lambda^2}, \qquad a(t) = \sqrt{\sigma_A^2 t + \lambda^2},$$

$$B(d, a) = \Phi\Big(\frac{\ln d}{a} - \frac{a}{2}\Big) - d\,\Phi\Big(-\frac{\ln d}{a} - \frac{a}{2}\Big).$$

Survival to t is $B(d, a(t))$. Note that $B(d, \lambda) < 1$. The firm has some chance of being past the barrier today, because the barrier is uncertain.

CDS spread. Set $z = \sqrt{\frac{1}{4} + 2r/\sigma_A^2}$ and

$$G(d, x, z) = d^{z+\frac{1}{2}}\Phi\Big(-\frac{\ln d}{x} - zx\Big) + d^{\frac{1}{2}-z}\Phi\Big(-\frac{\ln d}{x} + zx\Big),$$

$$H(T) = e^{r(\lambda/\sigma_A)^2}\big[G(d, a(T), z) - G(d, \lambda, z)\big].$$

$$c(T) = \frac{360}{365}(1 - R)\,\frac{1 - B(d, \lambda) + H(T)}{\big[B(d, \lambda) - B(d, a(T))e^{-rT} - H(T)\big]/r}.$$

Settings. Workbook: $\bar{L} = 0.5$, $\lambda = 0.3$, $R = 0.4$, $T = 5$, r the 5-year swap rate. Yu: $\lambda = 0.3$, $R = 0.5$, with $\bar{L}$ fitted.

Code. `_b`, `_g`, `_terms`, `model_spread`, `implied_asset_vol` in `e2c_model.py`. `survival` and `cds_par_spread` in `e2c_bond.py`. `fk_survival` in `fk_model.py` is the same survival formula with a shifted barrier $B = \bar{L}D$.

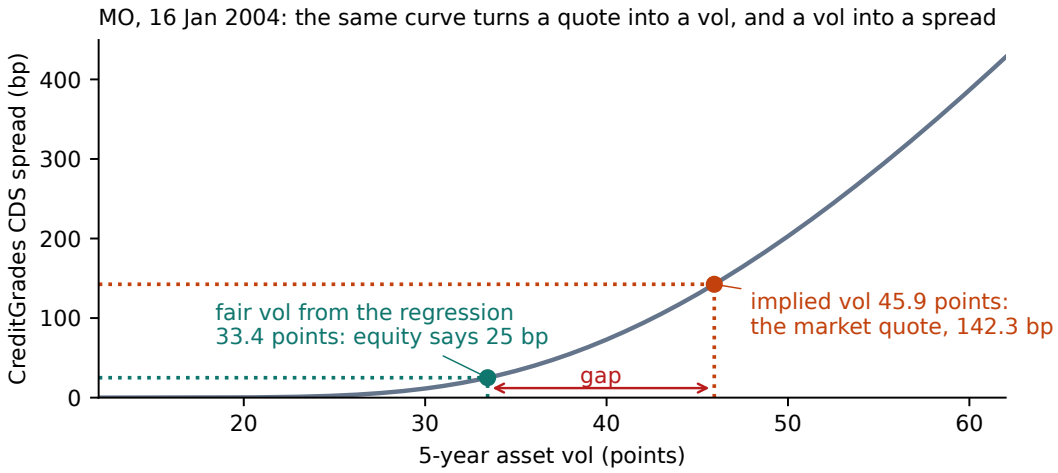


Figure 2: One curve, read in both directions. Read up from a vol to get a spread. Read across from a quote to get a vol. The curve is steep at high vol and flat near zero, so a spread gap and a vol gap do not measure the same thing.

3 The workbook method

In plain English

What it is. This is your own 2005 model, rebuilt in Python. It runs every Friday on a list of names with 5-year CDS. It has eight steps.

1. Turn the balance sheet into debt per share.
2. Average several equity vols and convert them to asset vol. Call this the **historical** vol.
3. Find the asset vol that makes the formula match the CDS quote. Call this the **implied** vol.
4. Compare implied with historical across all names that week. The ratio changes with leverage, so fit a line through the cross-section.
5. Use that line to get each name's **fair** vol.
6. Compute the gap between implied and fair. This is the **residual**.
7. Trade if the residual passes ±6. Size the trade so a 3-point move earns a fixed dollar amount.
8. Hedge with stock, mark to the next week, and rebuild the book.

Why it matters to you. This is the only method with real data behind it. It gives the 2.51 Sharpe. It rebuilds the workbook: the repository logs 11,465 comparisons with no failures.

The catch. The model compares each name with its peers that week. So it asks "rich against peers", not "rich against the true value".

Equations and code, in `e2c_model.py`

S01 Debt per share (`debt_per_share`). The 0.5 weight treats half of other liabilities as debt.

$$F = \text{st} + \text{lt} + 0.5(\text{other}_{st} + \text{other}_{lt}), \quad k_0 = \min(\text{minority}, 0.5F), \quad k_1 = \min(\text{pref}, 0.5\,\text{mcap}),$$

$$D = \frac{F - k_0}{(\text{mcap} + k_1)/S}.$$

S02, S03 Vols (`average_equity_vol`, `historical_asset_vol`). The equity vol is the plain average of the 30, 60, 90, 180 and 360-day realised vols and the 3, 6, 12 and 24-month put vols. Then $\sigma^{\text{hist}} = \sigma_S\, S/(S + 0.5D)$.

S06 Implied vol (`implied_asset_vol`). Solve $c(5; \sigma^{\text{imp}}) = c^{\text{mkt}}$.

S07 Regression (`leverage_regression`). On each date, across all names,

$$\ln \frac{\sigma_i^{\text{imp}}}{\sigma_i^{\text{hist}}} = \ln w_t + \beta_t \ln \frac{S_i}{D_i} + \epsilon_i.$$

S08 Fair vol (`fair_asset_vol`). With $p = 0.95$, prior coefficient 1 and prior skew 0,

$$\sigma_i^{\text{fair}} = \sigma_i^{\text{hist}} \Big[p\, w_t (S_i/D_i)^{\beta_t} + (1-p) \Big].$$

S09 Residual (`vol_residual`). $e_i = 30 \ln(\sigma_i^{\text{imp}} / \sigma_i^{\text{fair}})$.

S11 Signal (`trade_signal`). $e_i > 6$: sell protection, short stock. $e_i < -6$: buy protection, long stock.

S13 Size (`trade_size`). ν is the CDS value change per vol point, Q the notional in \$M. The 3-point target earns \$0.2M, with a \$10M limit.

$$Q = \min\Big(10,\ \frac{0.2}{|\nu| \cdot 3 \cdot \sigma^{\text{imp}}/30}\Big).$$

S14 Hedge (`equity_hedge`). Stock hedge in \$M is $Q\Delta$, with Δ from `cds_greeks`.

S15 P&L (`weekly_pnl`), in \$ thousands, for sold protection. A is the risky annuity, δ the days to the next row, ρ the stock return.

$$\underbrace{-Q\,\Delta c\,A/10}_{\text{spread}} \underbrace{-\,Q\Delta\,\rho \cdot 1000}_{\text{stock}} \underbrace{+\,Q\,(c/10)\,\delta/360}_{\text{carry}} \underbrace{+\,Q\Delta\,(r-0.5\%)\,\delta/360 \cdot 1000}_{\text{short rebate}}.$$

A purchase of protection flips the signs and pays $r + 0.5\%$ on the long stock.

Worked example on a real row: MO, 16 January 2004 (the row `05_io_spec.md` uses)

Step	Computation	Result
Debt per share	$F = 3376{+}21163{+}0.5(14819{+}10672) = 37284.5$. $k_0 = 4760$. $k_1 = 0$. Shares $= 109033.3/53.68$	$D = 16.01$
Equity vol	Average of the nine vols	24.10
Historical asset vol	$24.10 \times 53.68/(53.68 + 8.01)$	20.97
Implied asset vol	Solve the formula at 142.3 bp, $r = 3.41\%$	45.94
Regression that date	$w_t = 1.1409$, $\beta_t = 0.2929$. $S/D = 3.352$	
Fair vol	$20.97\,[0.95 \times 1.1409 \times 3.352^{0.2929} + 0.05]$	33.44
Residual	$30 \ln(45.94/33.44)$	9.52
Signal	$9.52 > 6$	sell protection
Size	$\nu = 0.00627$. $0.2/(0.00627 \times 3 \times 45.94/30)$	\$6.94M
Stock hedge	$6.94 \times \Delta$, with $\Delta = 0.1025$	\$0.711M short
Spread leg	142.33 to 136.25 bp, $A = 4.510$	+19.04
Stock leg	53.68 to 54.48, a loss on the short	−10.60
Carry, 7 days	$6.94 \times 14.233 \times 7/360$	+1.92
Short rebate	$0.711 \times (3.41\% - 0.5\%) \times 7/360 \times 1000$	+0.40
Week's P&L	\$ thousands	**+10.76**

Read it this way. The credit tightened and earned money. The stock rose, so the hedge lost part of it. One good week proves nothing. It only shows the arithmetic.

Whole-panel result, from `run_pipeline`

19,489 rows, 170 names, 151 Fridays from 27 September 2002 to 23 September 2005. Gross Sharpe 2.51, annual return 20.6%, annual vol 8.2%, worst drawdown 3.6%, $t = 4.28$. These equal the workbook's cells. They count no trading costs and assume you trade at the signal's own close.

4 Yu's method on bonds

In plain English

What it is. Fan Yu (2005) tested the same idea on CDS. Your files have no bond version, so the repository adapts Yu to bonds. The bond's market spread takes the place of the CDS quote.

The main differences from the workbook.

- It compares each name with its own model value. It does not compare it with peers that week.
- It fits one number for each name, the barrier, on the first days of data. It then freezes that number. So the first days set the "normal" gap for that name.
- It checks every day and enters when market and model differ by a set ratio.
- It exits when the two meet, when the holding period ends, or when the position's value goes negative.

Why it matters to you. Bonds are what you can compare with the published bond results. But a bond also carries interest-rate risk and has thinner trading, so this method adds a Treasury hedge and trade cleaning.

New terms. A **Z-spread** is the constant spread over the Treasury zero curve that prices the bond's cash flows at its market price.

Equations and code, in `e2c_bond.py`

Market spread (`z_spread`). Solve $P^{\text{full}} = \sum_i CF_i\, DF(t_i)\, e^{-z t_i}$ for z^{mkt}.

Model price (`risky_bond_price`), an adaptation. Recovery $R_b = 0.5$ of face is paid at default.

$$P^{\text{model}} = \sum_i CF_i\, DF(t_i)\, q(t_i) + R_b\Big(1 - q(0) + \int_0^T DF(s)\big(-q'(s)\big)\, ds\Big).$$

Solving the Z-spread equation on P^{model} gives z^{model} (`model_z_spread`).

Barrier fit (`calibrate_barrier`, `fit_barrier_on_bond`). Choose one $\bar{L}$ by least squares on the spread over the first days. Yu uses 10 days of CDS. The bond default is 20.

$$\hat{\bar{L}} = \arg\min_{\bar{L}} \sum_{t \le 20} \big(z_t^{\text{model}}(\bar{L}) - z_t^{\text{mkt}}\big)^2.$$

Signal (`entry_signal`, Yu Eq. 8). $\alpha = 0.5$ by default.

$z^{\text{mkt}} > (1+\alpha) z^{\text{model}} \Rightarrow$ long bond, short stock. $\quad$ $z^{\text{model}} > (1+\alpha) z^{\text{mkt}} \Rightarrow$ short bond, long stock.

Legs (`build_position`). Stock hedge per 1 of face is $\partial P^{\text{model}}/\partial S$ (`bond_equity_delta`), holding asset vol fixed. A Treasury hedge matches dollar duration (`rate_hedge_face`).

Exits (`exit_reason`, `bond_exit_reason`). Value below zero, convergence, end of the 180-day hold, plus forced events. Capital is 0.5 per 1 of notional, and execution lags the signal by one day (`BacktestConfig`).

Additions beyond Yu. Trade cleaning, a peer adjustment, a sibling and CDS veto, a cost gate. The file tags these [ADAPTATION]. `run_bond_backtest` is the entry point, and `run_yu.py` is the one-command runner.

Worked example. ALL INPUTS ARE MADE UP (`worked_example.py`)

A made-up 5.5% bond with 4.8 years left, and a made-up issuer.

1. Fit: 20 days with market Z near 171 bp give $\bar{L} = 0.8410$, with a fit error of 2.3 bp.
2. Later, the bond sells off. Market Z is 329.9 bp and model Z is 197.0 bp.
3. The ratio is 1.674, above 1.5. The signal is long bond, short stock, short Treasury.
4. Good path: net +8.69% on capital. Bad path, same entry: net −21.04% on capital.

Read it this way. The author chose both paths. The example shows the arithmetic and gives no evidence on the trade.

5 Finkelstein: options join in

In plain English

What it is. Finkelstein's 2002 deck uses the same structural idea and calibrates it to equity options. Options set the asset vol at each maturity. The same model then prices stock, options, CDS and bonds.

What is new.

- A put on a risky stock is partly default insurance. So the put market holds its own view of credit.
- With three markets you get three gaps. Two of them are independent. They can say *which* market is out of line.
- Because one model prices everything, you can solve a hedge against stock moves, vol moves and default at once.

Important. The deck gives no arbitrage signal and no trading rule. It has no entry, exit, sizing, costs or backtest. Every trading rule built on it in this repository is an addition, and the code tags these [ADAPTATION].

Why it matters to you. Agreement across markets is the best untested idea for picking names better. It needs dated option quotes, which the old panel does not have.

State of the code. The model, values and hedge work are coded and tested against the deck's numbers. No option adapter or runner exists yet.

Equations and code, in `fk_model.py`, `fk_value.py`, `fk_risk.py`

Shifted lognormal stock with a knock-out. Assets V have zero drift and start at $S_0 + B$. The stock is

$$S_t = (V_t - B)\, e^{(r-p)t}$$

until V touches the barrier B. After that the stock is zero.

Survival (`fk_survival`). With no barrier uncertainty,

$$Q(T) = \Phi\Big(\frac{\ln d}{A} - \frac{A}{2}\Big) - d\,\Phi\Big(-\frac{\ln d}{A} - \frac{A}{2}\Big), \quad d = \frac{S_0 + B}{B}, \quad A = \sigma(T)\sqrt{T}.$$

With uncertainty λ, d gains a factor e^{λ^2} and A^2 gains λ^2.

Vol term structure (`forward_vol`, `flat_forward_sigma`, `equity_vol_from_asset`). Option vols at each expiry give the cumulative asset vol $\sigma(T)$, using $\sigma_S = \sigma_A(S + B)/S$. Forward vol between two expiries is $\sqrt{(\sigma_2^2 T_2 - \sigma_1^2 T_1)/(T_2 - T_1)}$.

Put (`fk_put`). The deck prints no put formula. The code derives one from the reflection principle.

$$\text{put} = \underbrace{K e^{-rT}\big(1 - Q(T)\big)}_{\text{default part}} + \underbrace{e^{-pT}\Big[G(V_0) - \frac{V_0}{B} G\big(B^2/V_0\big)\Big]}_{\text{diffusion part}},$$

where $G(x) = E[(H - V_T)^+;\, V_T > B]$ for lognormal V_T from x, $V_0 = S_0 + B$ and $H = B + Ke^{-(r-p)T}$.

Three gaps (`three_gaps`), an adaptation. z_m is the market bond spread, z_o the option-implied model spread, z_s the stock-only model spread.

$$G_{bp} = \ln \frac{z_m}{z_o}, \qquad G_{ps} = \ln \frac{z_o}{z_s}, \qquad G_{bs} = \ln \frac{z_m}{z_s} = G_{bp} + G_{ps}.$$

Hedge (`solve_hedge`). Pick a row for each hedge leg from delta, vega and jump. Solve $Mn = -g$, where g holds the target's sensitivities and M the legs'. The guards reject a short option and a very large stock position. Those guards are additions.

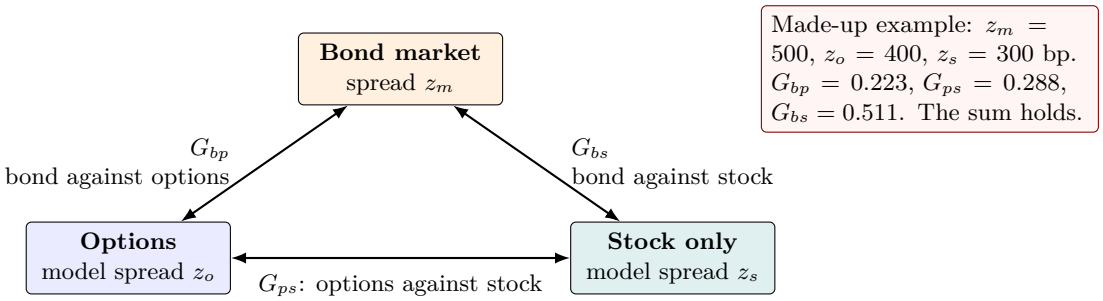


Figure 3: Three markets, three gaps, two independent. If only G_{bp} is large, the bond looks out of line. If only G_{ps} is large, the bond and options agree against the stock.

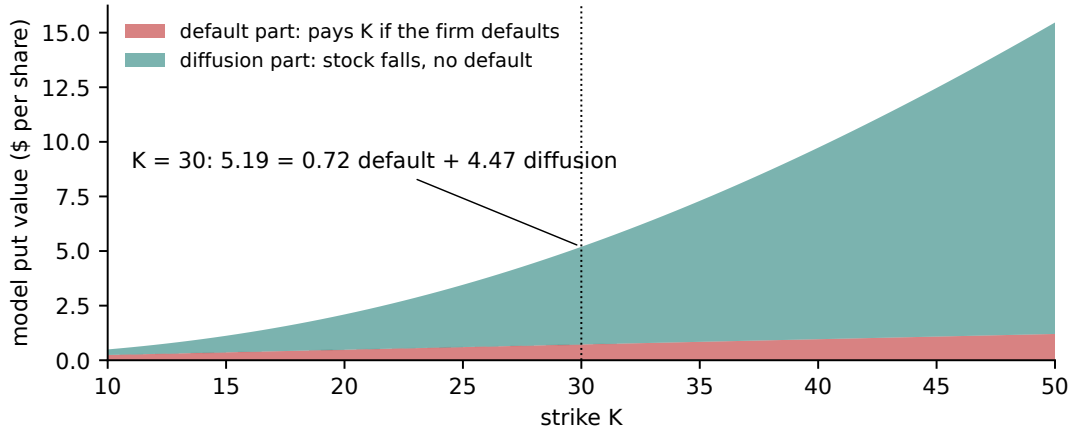


Figure 4: A put splits into two parts, from `fk_put`. The setting is that of slides 21–25: $S = 40$, $B = 15$, $\sigma = 0.375$, $r = 5\%$, $p = 2\%$, two years. The deck quotes this put at 5.22, and the code computes 5.19.

Worked hedge from the deck's setting (`fk_risk.py`, per $100 protection sold)

Sell $100 of 5-year protection at 560 bp with $R = 10\%$. Hedge with the $K = 30$ two-year put at 5.22 and with stock. Protect against stock moves and against a sudden default.

	Delta	Jump	Vega
Protection sold	+0.657	−90.00	−53.29
One put	−0.227	+21.93	+8.64
Hedge: 4.96 puts ($25.89) and 0.469 shares ($18.76)			
Net after the hedge	0	0	−10.44

Three things to notice. The stock position is long, not short. Most of the default protection comes from puts. Vega is left over, because two legs can only cancel two risks.

A convention to know. This table leaves the contract's own value out of the jump, which is the code's default and fits the deck's slides best. Setting `include_value=True` gives a jump of −95.51, then 5.39 puts and 0.566 shares. The note `model_analysis_and_improvements` quotes that version.

Deck numbers. Slide 1 says a put splits 1.90 of 4.92 at the money and 1.42 of 2.72 at 30% out of the money. Those values do not all reproduce from the slide's own inputs. The plan lists the differences.

6 The debt-equity spread and F10

In plain English

What it is. Chen, Chen and Li (2026, *Journal of Finance*) measure the gap between a bond's actual spread and its equity-implied spread. They call it the debt-equity spread (DES). Each month they rank firms into five groups. Bonds in the top group, the widest against equity, then outperform. Stocks in that group underperform.

What F10 is. F10 is an acceptance test in the plan. It asks: on your data, do you see the published pattern? It is the first test that needs real data and no options.

Why it matters to you. It gives an outside benchmark. If your data and code cannot show the published signs, you stop and debug before any option work.

Important. No copy of the paper is in this repository's working files. The published figures and definitions below come from the plan's notes on it. So a miss may reflect a definition gap and not a failed idea.

State of the code. This section describes commit `784361b` on branch `method/f10`. That work was still running when this was written. F10 has not run on real data, so it has no result.

Equations and code, in `fk_des.py`, `fk_f10.py`, `run_f10.py`

The gap (`bond_des`). Each spread is a yield less the yield of a synthetic Treasury with the same cash flows. The Treasury cancels.

$$\text{DES} = CS_D - CS_E = y(P^{\text{mkt}}) - y(P^{\text{equity model}}).$$

Equity model (`des_survival`). Extended CreditGrades. Assets drift at $r - \delta$, and barriers above today's assets are cut off. $\bar{L} = 0.5$, $\lambda = 0.3$, recovery 0.35 of face.

Inputs (`payout_delta`, `rolling_equity_vol`, `accounts_asof`). Payout δ is dividends, interest and repurchases over the sum $S + D$ per share, capped at 15%. Vol is 3-year realised. Accounts lag at least two months.

Firm and rank (`firm_des`, `quintile_sort`). Average DES across a firm's bonds, weighted by lagged bond value. Sort into quintiles each month.

Book (`t0_returns`). Long top-group bonds, short bottom-group bonds. Long bottom-group stocks, short top-group stocks. Holds of 1, 3 and 6 months. Each leg is reported apart.

Added for F10. Bond screens with a funnel (`screen_bonds`). Financing and borrow

fees (`financing_drag`). A Treasury overlay (`overlay_series`). Splits by bond size and fee (`median_split`). The plan's stop rule on large bonds (`stop_rule`). Newey–West t-statistics with 6 lags (`newey_west`).

Point in time. `run_f10.py` builds each month's inputs as known at the end of that month (`build_vintage_inputs`). A later correction cannot reach an earlier sort.

The test

Published targets, from the plan. These are gross. Window for the main test is January 2002 to December 2021, value-weighted, 1-month hold.

Bond premium, full sample	5.28% a year, $t = 5.64$
Bond premium, clean returns, 2002–2021	3.24% a year, $t = 2.69$
Stock premium	6.56% a year, $t = 3.79$
Quintile gaps, Q1 to Q5	−139 to +271 bp
Price ratio, Q1 to Q5	0.94 to 1.16

The plan's rule. "Signs must match. Sizes as shape, if samples differ." So: Q1 gap below 0, Q5 gap above 0, Q1 ratio below 1, Q5 ratio above 1, bond premium above 0, stock premium above 0.

Rules added in `fk_f10.py`, written before any real data was seen.

1. The bond premium also needs $t \geq 1.96$, because a sign on noise is no reproduction.
2. NOT JUDGED below 60 months or a median of 25 sorted firms.
3. Both continuous and semiannual yields run. You have not chosen, so the overall verdict is the weaker of the two.
4. The size gap to 3.24% is reported and does not decide the verdict.

Verdicts. REPRODUCED, NOT REPRODUCED, or NOT JUDGED with a reason.

What a REPRODUCED verdict would not mean

It means the signs match on your sample. It does not mean you could earn 3.24%. The published figures are gross, and the plan's stop rule looks at the net return on large bonds. A clean result still depends on the point-in-time data and on dead names being in the universe.

7 Selecting names: the hold band and the cap

In plain English

The problem. The workbook rebuilds its book every Friday. A name that drifts from 6.1 to 5.9 leaves the book, and it may come back next week. Each round trip pays the spread. At a 5% round-trip cost, costs take most of the return.

The hold band. Enter past 6. Stay in until the gap falls to 3. This stops the churn from small wobbles around the trigger.

The cap. Hold at most 20 names on each side. Fill free slots with the names that have the highest expected gain per dollar of cost.

What we learned.

- The hold band roughly doubles the Sharpe after costs, from 0.55 to 1.15.
- The cap keeps about half the names, at a Sharpe of 0.96. That is lower than the band alone, and five names matter more.
- A bigger gap does not pick next week's winners. The ranking works by avoiding cost.

Why it matters to you. The band is the base case. The cap is the alternative to compare on real

data. Both are off by default, so the workbook's 2.51 still reproduces.

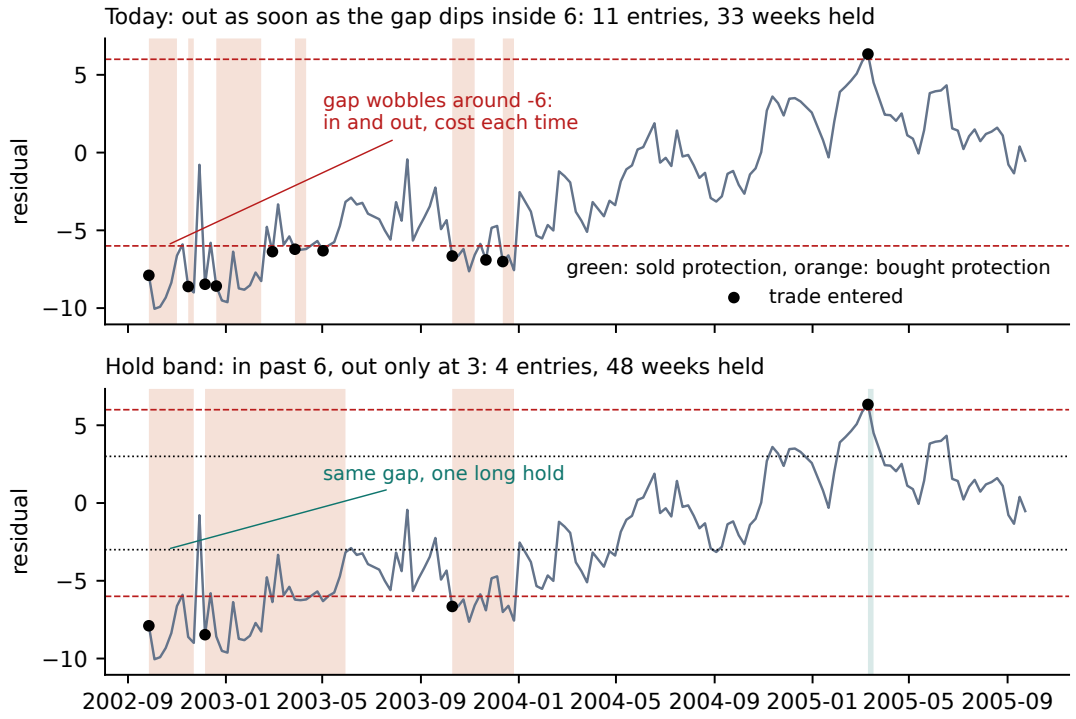


Figure 5: Before and after on one real name, GE. Each dot is a new trade, and each one pays the cost. Shading shows when a position is on. Today it enters 11 times over 33 weeks held. The band enters 4 times over 48 weeks held. I picked GE because the band changes it most among names with trades on both sides, so it shows the rule and is not typical.

Rules and code, in `e2c_model.py`, step S11a

Hold band (`held_signal`, `Params.exit_trigger`). Side $s = \pm 1$.

$$\text{enter if } s\, e_{i,t} > 6, \qquad \text{stay while } s\, e_{i,t} > 3.$$

A week with no row keeps the position. A blank residual or an out-of-range spread closes it.

Cap (`Params.max_per_side`). Held names keep their slots. New names fill free slots by

$$\frac{\text{gain}}{\text{cost}} = \frac{|\nu|\,(\sigma^{\text{imp}}/30)\,|e_i|}{A\,(c/10^4)\,b}, \qquad b = 5\% \quad (\texttt{gain_per_cost}).$$

Notional cancels, so the rank does not depend on the size limit.

Cost (`trading_cost`). For each change in signed size, in $ thousands,

$$|\Delta Q| \cdot 1000 \cdot A \cdot (c/10^4) \cdot b/2.$$

It covers CDS only. It has no stock cost and no execution lag, and the last open position pays no exit cost.

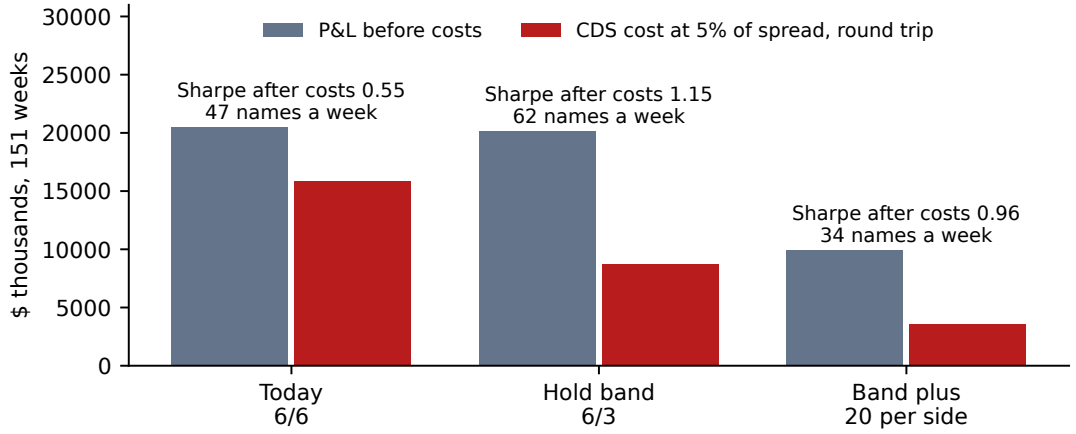


Figure 6: Where the money goes. The band gives up almost no P&L before costs and pays about half the cost. The cap roughly halves the P&L and cuts the cost by about 60%.

The numbers. All in-sample, on survivors, 151 weeks, cost 5% of spread

Book	Names	P&L	Cost	Sharpe, raw	Sharpe, NAV	Top 5 share
Today, 6/6	47	20,478	15,870	0.55	0.57	37%
Hold band, 6/3	62	20,145	8,711	1.15	1.15	36%
Band plus 20 per side	34	9,898	3,570	0.96	1.04	51%

P&L and cost are in $ thousands. Names is the average held per week.

Two Sharpe bases. "Raw" uses the weekly dollar P&L after cost. "NAV" divides by the week's NAV, the workbook's own return, and `run_workbook.py` reports this one. The gross figures are 2.49 raw and 2.51 NAV for today's book. Compare a number only with one on the same basis.

A correction. I first reported 1.15 for the capped book. That ranking gave every trade the full target gain. About 80% of trades sit at the $10M limit, so they cannot reach the target and the ranking overstated them. The corrected formula gives 0.96. The old ranking reproduces 1.15 raw, with a top-5 share of 47%.

Three checks, computed by `make_figs.py`

Does gap size pick winners? Among names past the trigger, the mean weekly rank correlation of $|e_i|$ with next week's P&L is 0.004, with $t = 0.31$. So it does not.

Does the cap beat chance? Twenty runs that fill the cap's slots at random give raw Sharpes from 0.38 to 0.89, with a mean of 0.64. The ranked book's 0.96 sits above all twenty. It sits below the uncapped band at 1.15. This run replaces an earlier one with a range of 0.47 to 0.86, which used the first ranking's study script and other random draws.

Is the book only riding the market? I regressed weekly P&L on the panel's mean stock return and median spread change, with Newey–West errors over 4 lags. About 85% of the mean P&L remains unexplained, and the Sharpe after removing these moves is 2.17. Each side alone is about half market risk (R^2 0.50 and 0.58). What remains is significant for short credit ($t = 2.57$) and not for long credit ($t = 1.21$). The two sides offset, so the check needs a balanced book. The market measures come from the panel's own survivors.

How much to trust this

I tried many rules on the same 151 weeks. Fix a rule before you see new data, and do not pick the best row. A rule that wins here can lose on real data with dead names and filing dates.

8 The data layer

In plain English

The problem. Your production data sits on another machine, and this public repository must not hold it. The methods also must not see data before it was public.

The answer. All data passes through one small door, a single `read` function. Your production code stays off this repository and only has to answer that one call. A **view** stands between the data and the methods and shows only what was known at a chosen time.

Why it matters to you. You can plug in production data by changing one setting, without changing any method. You can export files or write a small adapter. A check then runs first and tells you what is wrong with the data.

New terms. Point-in-time means as it was known on that date. **Look-ahead** is the error of using something before it was public. A **restatement** is a later correction to reported numbers.

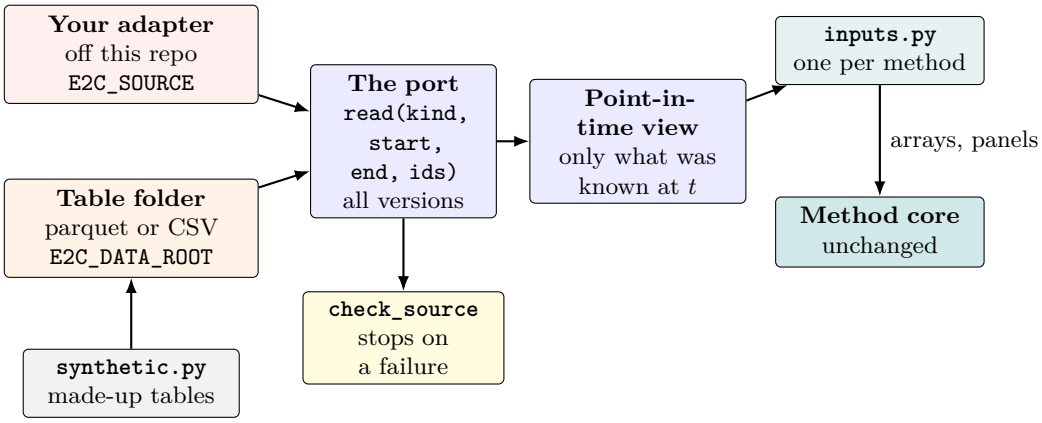


Figure 7: The flow. Sources sit on the left. Methods sit on the right. The method core knows nothing about where data comes from.

Interfaces and code, in `e2c_data/`

Port (`source.py`). Any object with this method is a source. It returns every version it holds.

```
read(kind, start=None, end=None, ids=None) -> DataFrame
```

View (`view.py`). `PointInTimeView(source, decision_time)` has three reads.

- `read`: the latest version of each row known at the decision time.
- `versions`: every version known at the decision time. A backtest needs this for accounts, or restated numbers would reach earlier dates.
- `issuers_on`: dated instrument-to-issuer links. Two issuers at once raises an error.

A read past the decision time raises `LookAheadError`. A missing kind raises `DataUnavailable`. Missing values stay NaN, never zero.

15 kinds (`schema.py`). Each carries `available_at`, the moment it became public.

Identity	`security_map`, `universe`, `issuer_attributes`
Equity	`equity_prices`, `corporate_actions`
Accounts	`accounts`
Rates, funding	`rates`, `financing`
Credit	`cds_quotes`, `bond_terms`, `bond_prices`, `bond_trades`
Options, events, factors	`option_quotes`, `events`, `factor_returns`

Units. Rates, spreads and vols are decimals. Bond prices are per 100 of face. Amounts are USD. Each adapter converts to its method's units.

Contract (`contract.py`). `check_source` tests missing kinds, columns, keys, units, look-ahead, broken links and survivorship. It adds a warning when an end date is visible before it happened.

Plug-in (`factory.py`). `open_source` reads `E2C_SOURCE=package.module:function`, or else uses the folder `E2C_DATA_ROOT`. Only this module reads the environment.

Why a view needs two reads: a restatement

A firm files a figure on 14 February. It restates it on 1 March. A backtest viewed from the last date with `read` would use the restated figure in February too. That is look-ahead. With `versions`, the method sees both and picks by date.

A test on made-up data confirms this for `fk_des`. Results match from January to April, because neither run can use the restated number yet. They differ in May and June, once it is in use after the two-month lag. They match again in July, when the next filing replaces it.

9 What is proven and what is not

In plain English

The short answer. The arithmetic is well tested. The plumbing is tested. Whether the trade makes money is unknown. No Sharpe ratio in this document is evidence of live profit.

What	Data used	What it shows
Workbook pipeline	Real 2002–05 panel	Rebuilds the workbook. 2.51 reproduces
Workbook adapter	Real panel, invented release dates	The data layer leaves the panel unchanged, to within rounding. Same flags and Sharpe
Hold band, cap, cost	Real panel, in-sample	Ranks ideas. Not a forecast
Yu on bonds	Made-up only	The code runs and gives the expected trades
Finkelstein model	Deck's numbers, made-up	Matches most deck numbers. The plan lists the rest
DES and F10 runner	Made-up only	Planted signals are found. No real verdict exists
Production run	None yet	Nothing known

The limits that could change the answer

1. **Survivors only.** The panel has 170 names and no name that ever ended. `check_source` fails it for that reason. Dead names may make results worse.
2. **Invented release dates.** The panel has no release dates. The test converter stamps 21:00 on each row's date. Its balance sheets appear at period end, before filing. So the round trip proves the plumbing, not point-in-time correctness.
3. **Only accounts are versioned through time.** In the workbook and Yu runners, prices, CDS

quotes, links and rates use the latest version known at the decision time. So later corrections can reach earlier dates. Yu trades use their report times. `run_f10.py` avoids this by rebuilding each month.

4. **Hindsight tables.** `check_source` warns and does not fail when an end date shows too early, since a merger announced in advance looks the same. It can miss some exports, such as a dead name that first appears only in its end-date version.
5. **Costs and lag.** The band study charges CDS costs only. It has no stock cost and no execution lag. A one-week lag alone took the gross Sharpe from 2.51 to 1.47 in an earlier reconstruction.
6. **Short sample.** 151 weeks, in a period when stocks rose and spreads fell. Top ten names earned 55% of today's book's P&L.
7. **Same model everywhere.** Agreement among methods is weak evidence, because they share a formula.
8. **Vols on production data.** The workbook's vols arrive ready-made. If you derive them, you cannot claim to reproduce the workbook.

Where each number comes from

- Sections 1, 3, 7 and the figures: `notes/e2c_explained_figs/make_figs.py`, which writes `numbers.json`. It runs the code on `method/workbook`, commit `88db996`.
- Section 4: `02_bond_vol_method.md` on `method/yu-bonds`.
- Section 5: `fk_put`, `cds_seller_greeks`, `put_greeks` and `solve_hedge`, in the script above.
- Section 6: `fk_f10.py`, `F10.md` and the plan, on `method/f10`, commit `784361b`.
- Section 8: `e2c_data/` on `main`, commit `ac9a9a7`.